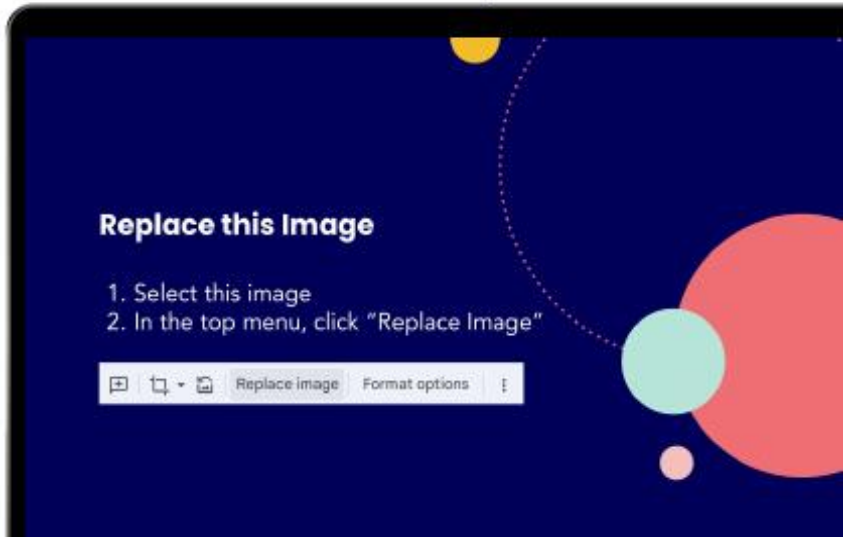
hum

MiniProject operating model

Fewer handoffs, faster launches

Adapting delivery for AI-accelerated work without turning every idea into a ticket cloud.



June 2026

hum

The old model worked

When implementation was the expensive part

The agile gold standard was to break work into the smallest testable units: frontend, backend, data, infra, design, analytics, QA.

Why it helped

Parallel work created leverage when each technical layer took meaningful time.

What it optimized

Card completion, independent testing, and predictable handoffs across specialists.

The catch

The model assumes implementation is slower than coordination.

hum

The economics changed

Implementation compressed. Coordination did not.

Same handoffs.

Smaller work.

Worse ratio.

Before

A 30-point body of work could justify several owners and many cards.

Now

AI may compress the same work into 13-ish points, but every owner still needs context, review, sequencing, and coordination.

Result

The process can become the bottleneck during the work, not just before it.

hum

What slows us down now

Not the code path. The coordination surface area.

Every extra card and owner adds seams: context sharing, dependency management, partial ownership, repeated explanation, review queues, and “who owns this?” moments.

Tiny cards are not free

A board full of small tasks can still carry a large coordination bill.

Parallelism is no longer automatic leverage

More hands can mean more handoffs when the underlying tasks are smaller.

The new target

Reduce seams during the work so ideas reach first launch faster.

hum

The shift

PRs stay small. Ownership gets bigger.

Old default

Split by technical layer: frontend, backend, infra, data, analytics, QA.

→

New default

Bundle by complete improvement: one coherent problem, one engineer, launch-ready by design.

We still use small PRs. We stop making the shared planning board carry every implementation crumb.

hum

MiniProject

Small enough for one engineer to own. Complete enough to ship.

A MiniProject is a complete, demoable improvement. It can touch frontend, backend, data, AI, tests, analytics, and deployment - but it has one owner and one coherent launch-ready result.

Complete

Not a partial technical layer.

Demoable

Something real to show.

Owned

One engineer carries the path.

Launch-ready

Safe to release, even if held.

hum

More autonomy, clearer responsibility

Engineers are problem owners, not ticket takers.

MiniProjects give engineers more room to operate. That room comes with responsibility for the whole path from problem to working improvement.

1. **understand the problem**
2. **shape the technical approach**
3. **choose how to use AI**
4. **break work into PRs**
5. **get feedback**
6. **test, QA, and manage release risk**

hum

The partnership model

Business defines value. Engineering bundles delivery.

Product / CSX / Business

Define the customer need, priority, urgency, launch goals, and what matters most.

Engineering

Defines MiniProject boundaries, sequencing, technical approach, testing, and release safety.

Stakeholders can request Projects, Discovery, or Chores. Engineering creates MiniProjects after the work has enough shape to bundle safely.

hum

The four work types

A shared map, not a new bureaucracy

Project

A larger initiative or product/business goal that may contain multiple pieces of work.

Discovery

Work to understand, decide, design, rank, or de-risk before build.

MiniProject

A complete improvement one engineer owns end to end.

Chore

A small, specific request with low coordination overhead.

Small requests get speed.

Broad requests get shaping.

Why specificity matters

The fastest path is a Chore

Specific ask

“Add the onsite optimization widget to the sales blog.”

Clear, bounded, low variance.

Broad ask

“Improve onsite optimization.”

Worth considering, but it needs Discovery or Project shaping before timing is predictable.

The broader the ask, the more Engineering must reason through bundles, dependencies, testing, release safety, and sequencing.

hum

Use the meeting points we already have

Same process map, clearer decisions

1	**Product Prioritization**	What Projects should Product focus on?
2	**Product Refinement**	What Discovery cards belong inside each Project?
3	**Tech Refinement**	Based on Discovery, what MiniProjects are in the Project?
4	**Tech Estimation**	What effort is associated with each Chore and MiniProject? Discovery is often time-bound.
5	**Sprint Planning**	What does the business want to put in the next sprint?

Why this matters

The model should fit our operating rhythm. We are not inventing a shadow workflow; we are making the decision points sharper.

hum

Example Project

Advanced Site Optimization Widgets

The Project is the container for sequenced work. It may include Chores, Discovery, and several Engineering-created MiniProjects.

Product Prioritization

Decide whether this Project should be a Product focus now. Compare against other Projects competing for attention.

Project goal

Create, place, target, measure, and improve onsite widgets.

Why it is a Project

It contains multiple launchable improvements and several questions that need Product, AI, and Engineering Discovery.

Project, not ticket cloud

The Project holds the map. MiniProjects carry delivery.

hum

Product Refinement

What Discovery cards are in the Project?

Product Refinement turns the broad Project into the specific questions that need answers before Tech can confidently bundle delivery.

Product	**Develop and rank user stories; determine v1 scope**
Product	**Develop UI for Inline Widget Builder**
Product	**Develop UI for On-Site Placement Selector**
AI + Product	**Determine effective engagement tracking for onsite optimization**
Optional CSX input	**Add customer examples, urgency, workarounds, and account impact**

Output

Discovery cards with owners and useful outputs, not a full technical breakdown.

hum

Tech Refinement

Based on Discovery, what MiniProjects are in the Project?

Tech Refinement uses Discovery output to bundle complete improvements one engineer can own end to end.

1. **What is complete and demoable?**
2. **What can one engineer own end to end?**
3. **What can be tested and made launch-ready?**
4. **What needs more Discovery first?**

MiniProjects likely created

A. Add Inline Widgets, CSS Selector Controlled
B. Inline Widget Reporting
C. Onsite Widget Placement Tool
D. Segment-Based Onsite Widget Variants
E. Recommendation Subwidget

What stays inside the owner plan

PR sequence, implementation details, tests, rollout steps, and escalation points unless they need team visibility.

Tech Estimation

Estimate Chores and MiniProjects; time-box Discovery

Chore

Add Onsite Optimization Widget to Blog for Sales Enablement.

Small, specific, low variance. Estimate normally.

Discovery

Technical path for reading/running customer sites.

Often time-bound: produce a decision, approach, or next step.

MiniProject

Inline Widget Reporting.

Estimate as a complete improvement: reporting behavior, tests, and release readiness.

The cliff:

Specific asks can stay in the fast lane. Broad asks move into Project sequencing, which is healthy but carries more timing variance.

Key rule

Do not estimate Discovery like normal implementation when the right output is learning, ranking, design, or de-risking.

hum

Sprint Planning

What does the business want in the next sprint?

Sprint Planning chooses from ready work: Chores, time-bound Discovery, and estimated MiniProjects.

Fast lane

Pull the Chore when it is useful and specific.

Learn lane

Pull Discovery when the next decision needs a time-bound answer.

Build lane

Pull MiniProjects when the bundle is complete enough for one owner to make launch-ready.

The board should show complete improvements and decision work. The owner plan can carry PR sequence, test details, rollout mechanics, and other implementation crumbs.

Outcome

Fewer projects stuck in refinement purgatory. More complete improvements reaching first launch.

hum

What this is not

Keep the guardrails. Drop the drag.

- **Not giant PRs**
- **Not less review**
- **Not less Product input**
- **Not engineers working alone**
- **Not every request becoming a Project**
- **Not process for process sake**

The goal is not to do less planning. It is to spend less time coordinating fragments and more time launching complete improvements.

hum

Less refinement purgatory.

Faster first launch.

The new promise: fewer handoffs, clearer ownership, and more complete improvements reaching users sooner.

Takeaway

MiniProjects are complete improvements: one owner, one coherent path, launch-ready by design.